Introduction

Lino Galiana
Version du 2 novembre 2026

Cette introduction présente l'objectif du cours,les partis-pris pédagogiques, le fil conducteur de cet enseignement ainsi que les modalités pratiques de celui-ci.

⚠️ **PDF généré automatiquement.** Ce document est produit à partir du site du cours et peut présenter des problèmes de mise en forme. Pour une version à jour et adaptée à la lecture, consultez la page du cours : pythonds.linogaliana.fr.

Table des matières

1 Introduction **2**
2 Pourquoi faire du `Python` ? **3**
3 Pourquoi faire du `Python` pour l'analyse de données ? **3**
4 Pourquoi apprendre `Python` quand il existe des IA génératrices de code ? **5**
4.1 Car coder, c'est résoudre un problème d'ingénierie 6
4.2 Car coder, c'est réflechir 6
4.3 Car coder, c'est actualiser, encore et toujours, ses connaissances 7
5 Objectifs du cours **7**
5.1 Initier à la démarche de la *data science* 7
5.2 Reproductibilité 8
5.3 Évaluation 9
6 Plan du cours **9**
Références **13**
Informations additionnelles 13
Bibliographie **15**

Compétences à l'issue de ce chapitre

- Comprendre pourquoi Python est devenu incontournable dans le domaine de la *data science* et du *data engineering*, notamment grâce à sa simplicité, sa lisibilité et son écosystème communautaire riche ;
- Identifier la manière dont Python peut vous assister dans les différentes étapes d'un projet de valorisation de données : structuration, exploration, modélisation, communication de résultats... ;
- Appréhender l'importance d'une démarche reproductible, rigoureuse et scientifique dans la conduite de projets de *data science* ou de *data engineering* et les moyens pour cela (*notebooks Jupyter*, open data, environnements standardisés...) ;
- Comprendre l'objet et les partis-pris pédagogiques du cours ainsi que son fil conducteur.

1 Introduction

Ce cours rassemble l'ensemble du contenu du cours ***Python pour la data science*** que je donne à l'ENSAE depuis 2020 et à l'ENSAI depuis 2026. Cela représente plus de 300 élèves qui suivent ce cours chaque année.

> **i** Version internationale
>
> L'année 2024 a permis l'arrivée progressive d'une version anglophone équivalente à la version française visant à servir de cours d'introduction à la *data science* pour les instituts statistiques européens suite à un appel à projets européen.

Ce site (pythonds.linogaliana.fr/) est le point d'entrée principal du cours. Il rassemble l'ensemble des contenus faits en cours dans le cadre de travaux pratiques ou proposés en complément à des fins de formation en continue. Ce cours est *open source* et j'accueille avec plaisir les suggestions d'amélioration sur `Github` ou par le biais des commentaires en bas de chaque page.

A partir de la rentrée 2026, ce cours connaît un changement important avec une division en deux partie, répartie sur deux semestres de scolarité, permettant d'approfondir chacune:

- Python pour la *data* au premier semestre correspondant à la première partie du cours consacrée à l'exploration de données ;
- Python pour le *ML* au deuxième semestre correspondant à la seconde partie du cours consacrée à la modélisation.

`Python` étant un langage vivant et très dynamique, les pratiques évoluent et ce cours s'adapte en continu pour tenir compte de l'écosystème mouvant de la *data science*, en essayant néanmoins de distinguer les évolutions pérennes des pratiques des effets de mode.

Si ce cours s'adresse en priorité à des étudiants en *data science*, il est suffisamment général pour intéresser de nombreux profils, étudiants comme professionels, issus de diverses formations académiques. Il est pensé pour être une introduction à la *data science* par le biais de Python.

> Pour aller plus loin
>
> Quelques éléments supplémentaires sont disponibles dans les slides introductives. Des éléments plus avancés sont présents dans un autre cours consacré à la mise en production de projets *data science* que je donne avec Romain Avouac en dernière année de l'ENSAE (ensae-reproductibilite.github.io/website) et dont une variante sera donnée prochainement à l'ENSAI.

> **i** Architecture du site web
>
> Ce cours présente des tutoriels et des exercices complets qui peuvent être lus depuis ce site ou édités et testés dans un environnement interactif de type *notebook* (voir prochain chapitre pour plus de détails).
>
> Chaque page est structurée sous la forme d'un problème concret et présente la démarche générique pour résoudre ce problème général. Tous les exemples s'appuient sur de l'*open data* et sont reproductibles.
>
> Vous pouvez naviguer dans l'architecture du site via la table des matières ou par les liens vers le contenu antérieur ou postérieur à la fin de chaque page. Certaines parties, notamment celle consacrée à la modélisation, proposent des exemples fil-rouge pour illustrer la démarche, et les différentes approches possibles d'un même problème, de manière plus extensive.

2 Pourquoi faire du `Python` ?

`Python`, dont le logo assez reconnaissable prend la forme de , est un langage qui a déjà plus de trente ans mais qui a connu, au cours de la décennie 2010, une nouvelle jeunesse du fait de l'engouement autour de la *data science*.

`Python`, plus que tout autre langage informatique, réunit des communautés aussi diverses que des statisticiens, des développeurs, des gestionnaires d'applications ou d'infrastructures informatiques, des lycéens - `Python` est au programme du bac français depuis quelques années - ou des chercheurs dans des champs à la fois théoriques et appliqués.

Contrairement à beaucoup de langages informatiques qui fédèrent une communauté assez homogène, `Python` est parvenu à réunir largement grâce à quelques principes centraux : la lisibilité du langage, la simplicité à utiliser des modules, la simplicité à l'associer à des langages plus performants pour certaines tâches données, l'énorme volume de documentation disponible en ligne... Être le deuxième meilleur langage pour réaliser telle ou telle tâche peut ainsi être une source de succès lorsque la concurrence ne dispose pas d'un éventail aussi large d'avantages.

Le succès de `Python`, de par sa nature de langage couteau-suisse, est indissociable de l'émergence du profil du *data scientist*, profil capable de s'intégrer à différents niveaux dans la valorisation de données. T. H. Davenport et D. Patil [1], dans la *Harvard Business Review*, ont ainsi pu parler du *« boulot le plus sexy du 21e siècle »* et ont pu, dix ans plus tard, faire un panorama complet de l'évolution des compétences attendues d'un *data scientist* dans la même revue [2]. Ce ne sont d'ailleurs pas que les *data scientists* qui ont vocation à pratiquer `Python` ; dans le halo des emplois autour de la donnée (*data scientist*, *data engineer*, *ML engineer*...), `Python` fait office de tour de Babel permettant la communication entre ces différents profils interdépendants[1].

La richesse de `Python` permet de l'utiliser dans toutes les phases du traitement de la donnée, de sa récupération et structuration à partir de sources diverses à sa valorisation. Par le prisme de la *data science*, nous verrons que `Python` est un très bon candidat pour assister les *data scientists* dans tous les aspects du travail quotidien leur permettant de tirer de la valeur de gisements hétérogènes de données. Ce cours a néanmoins l'ambition, peut-être illusoire, d'être plus qu'un énième cours d'introduction au langage `Python`: il s'agit plutôt d'un cours d'introduction à la *data science* s'appuyant sur `Python`. En fait, apprendre `Python` est un prétexte pour apprendre à avoir les bons réflexes quand on est confronté à un jeu de données.

Ce cours introduit différents outils qui permettent de mettre en relation, grâce à `Python`, des données à des concepts théoriques issus de la statistique ou des sciences économiques et sociales. Néanmoins, ce cours va au-delà d'une simple introduction au langage et revient régulièrement sur les apports, mais aussi les limites, du langage pour répondre à des besoins opérationnels ou scientifiques.

3 Pourquoi faire du `Python` pour l'analyse de données ?

Cette question est un peu différente car si `Python` est un langage commun pour découvrir la programmation informatique du fait de sa simplicité d'usage, comment se fait-il qu'il se soit imposé comme le langage dominant dans l'écosystème de la *data* et de l'IA ?

`Python` est d'abord connu, dans le monde de la *data science*, pour avoir fourni très tôt les outils utiles à l'entraînement d'algorithmes de *machine learning*, avant même que cette approche devienne incontournable. Certes, le succès de `Scikit Learn`, de `Tensorflow` ou plus récemment de `PyTorch` dans la

[1] Voir L. Galiana et O. Lefebvre [3] pour plus d'éléments de contexte sur l'évolution des emplois de la *data*.

communauté de la *data science* ont beaucoup contribué à l'adoption de `Python`[2]. Cependant, résumer `Python` à quelques librairies de *machine learning* serait réducteur tant il s'agit d'un véritable couteau-suisse pour les *data scientists*, les *social scientists*, les économistes ou plus généralement pour les praticiens de la donnée quelque soit leur champ d'application. La *success story* de `Python` n'est pas seulement le fait d'avoir proposé des librairies de *machine learning* à un moment adéquat: ce langage dispose de réels atouts pour de nouveaux praticiens de la donnée.

L'intérêt de `Python` est son rôle central dans un écosystème plus large autour d'outils puissants, flexibles et *open-source*. Il appartient, comme le langage , à cette classe de langages pouvant servir au quotidien pour des tâches très diversifiées. Dans de nombreux domaines explorés dans ce cours, `Python` est, de loin, le langage informatique proposant l'écosystème le plus complet et le plus simple d'accès. Contrairement à d'autres langages très populaires, notamment `JavaScript` ou `Rust`, la courbe d'appentissage de `Python` est très légère et on peut rapidement être opérationnel et produire du code de qualité, si on a les bons réflexes (que ce cours, ainsi que celui de mise en production, ambitionnent de donner).

En plus des projets d'IA (Note 1), `Python` est incontournable dès lors qu'on désire récupérer des données par le biais d'API ou de *web scraping*[3], deux approches que nous explorerons dans la première partie du cours. Dans les domaines de l'analyse de données tabulaires[4], de la publication de contenu web ou de la production de graphiques, `Python` présente un écosystème de plus en plus similaire à du fait de l'investissement croissant de `Posit` et sa traduction en `Python` des librairies qui ont fait le succès de dans le domaine de l'analyse de données (ggplot notamment)[5].

> Les **données tabulaires** sont des données structurées, organisées, comme leur nom l'indique, sous forme de tableau permettant de mettre en correspondance des observations avec des variables. Cette structuration se distingue d'autres types de données plus complexes: textes libres, images, sons, vidéos...

ℹ Note 1: Pourquoi parler si peu d'IA dans un cours de Python ?

[2] `Scikit Learn` est une librairie développée par les laboratoires de recherche publique français de l'INRIA depuis 2007. C'est un projet *open source* depuis le début. Le projet est désormais maintenu par `:probabl.`, une *startup* dédiée à la gestion du projet *open source* `Scikit` et tout son écosystème associé qui rassemble une partie des équipes de recherche de l'INRIA qui a a participé au développement de l'écosystème incontournable du *machine learning*.

`Tensorflow` est une librairie initialement développée par Google pour leurs besoins internes, celle-ci a été rendue publique en 2015. Bien que moins de moins en moins utilisée, notamment du fait de la popularité de `PyTorch`, cette librairie a eu une influence importante dans les années 2010 en favorisant l'usage des réseaux de neurone dans la recherche ou pour l'exploitation à des fins opérationnelles.

`PyTorch` est une librairie développée par Meta depuis 2018 et rattachée depuis 2022 à la *PyTorch foundation*. Il s'agit aujourd'hui du principal *framework* pour entraîner des réseaux de neurones.

[3] Dans ces deux domaines, le concurrent le plus sérieux pour `Python` est `Javascript`. Néanmoins, la communauté autour de ce dernier langage est plus orientée autour des problématiques de développement web que de *data science*.

[4] Dans le domaine des données non structurées, `Python` est le langage d'analyse hégémonique. Dans le domaine des données tabulaires, l'avantage compétitif de `Python` est moindre, notamment par rapport à , mais ces deux langages proposent un noyau de fonctionnalités assez similaires. Nous aurons l'occasion de régulièrement faire le parallèle entre ces deux langages lors des chapitres consacrés à la librairie `Pandas`.

Néanmoins, il ne s'agit pas, par ces éléments, de rentrer dans la guéguerre stérile vs `Python`.

Ces deux langages ayant beaucoup plus de points de convergence que de divergence, il est très simple de transposer les bonnes pratiques d'un langage à l'autre. Il s'agit d'un point qui est développé plus amplement dans le cours plus avancé que je donne avec Romain Avouac en dernière année d'ENSAE : ensae-reproductibilite.github.io/website.

[5] A terme, les *data scientists* et chercheurs en sciences sociales ou en économie utiliseront de manière presque indifférente, et en alternance, ou `Python`. Ce cours présentera ainsi régulièrement des analogies avec pour aider les personnes découvrant `Python`, mais connaissant déjà bien , à mieux comprendre certains messages.

Bien qu'une partie conséquente de ce cours évoque les problématiques de *machine learning* et d'algorithmie, je rechigne généralement à rentrer dans la mode, particulièrement forte depuis fin 2022 et la sortie de `ChatGPT`, de tout dénommer IA.

En premier lieu car le terme est souvent mal défini, galvaudé et instrumentalisé par des acteurs qui profitent de sa charge symbolique forte, lié à notre imaginaire de la science fiction, pour vendre un produit « miracle » ou au contraire activer nos peurs.

Mais aussi car ce terme recouvre énormément de méthodes potentielles si on accepte une définition large. Les parties modélisation et NLP de ce cours, celles les plus proches du sujet IA, se concentrent sur les méthodes d'apprentissage. Si on reprend les définitions suivantes S. J. Russell et P. Norvig [4] ou de l'IA Act Européen, on peut faire rentrer beaucoup plus que ces méthodes dans le concept d'intelligence artificielle:

> « L'étude des agents [intelligents] qui reçoivent des perceptions de l'environnement et agissent. Chacun de ces agents est mis en oeuvre par une fonction qui associe les perceptions aux actions, et nous couvrons différentes manières de de représenter ces fonctions, telles que les systèmes de production, les agents réactifs, les planificateurs logiques, les réseaux de neurones et les systèmes de gestion de l'information et les systèmes de théorie de la décision »
>
> S. J. Russell et P. Norvig [4]

> « Système basé sur une machine qui est conçu pour fonctionner avec différents niveaux d'autonomie et qui peut faire preuve d'adaptabilité après son déploiement, et qui, pour des objectifs explicites ou implicites, déduit, à partir des données qu'il reçoit, comment générer des résultats tels que des prédictions, du contenu, des recommandations ou des décisions qui peuvent influencer des environnements physiques ou virtuels »
>
> AI Act européen

Pour en apprendre plus sur ce sujet, voici une présentation que j'ai faite sur le sujet de l'IA en 2024:

Enfin, la question est également pédagogique. Quand on parle d'IA depuis 2023, on pense à l'IA générative. Je pense que pour comprendre le fonctionnement de cette approche, radicalement différent d'autres paradigmes, et être capable de mettre en oeuvre des projets d'IA générative porteurs de valeur, il faut avoir des notions sur les apports et les limites de l'approche scientifique du *machine learning*. Sinon, on se retrouve à construire des usines à gaz pour des besoins simples ou à être incapables d'évaluer l'apport d'une approche générative par rapport à d'autres approches. Ce cours étant introductif, j'ai donc choisi de me concentrer sur le *machine learning* et du NLP basique, là encore à un niveau assez introductifs mais néanmoins déjà un peu creusés, et laisser aux curieux le soin de s'autoformer sur les sujets genIA.

4 Pourquoi apprendre `Python` quand il existe des IA génératrices de code ?

Les assistants de code, notamment `Claude` et `ChatGPT`, ont changé à jamais le développement de code. La vitesse des changements est assez ahurissante.

Ces outils font maintenant parti des outils quotidiens des *data scientists* et offrent beaucoup de confort car ils peuvent, à l'aide d'instructions plus ou moins claires, générer du code `Python`. Et ces IA ayant étaient entraînées à partir de l'ensemble du code présent sur internet, et parfois spécialisées à répondre à des problèmes de développement, elles aident beaucoup.

L'intégration d'outils de *vibe coding* dans les IDE (notamment `Claude Code`) a encore accéléré ce processus et représente un réel défi pour le monde éducatif.

Puisque maintenant les IA génèrent du code, pourquoi continuer à apprendre à coder ?

4.1 Car coder, c'est résoudre un problème d'ingénierie

Car coder ce n'est pas juste produire des lignes de code, c'est se confronter à un problème, adopter une stratégie par étape pour y répondre, réfléchir à plusieurs solutions possibles et choisir la meilleure en arbitrant entre plusieurs objectifs (vitesse, simplicité, etc.), tester et corriger, etc. Le code est un outil au service d'un problème d'ingenieurie. Les IA savent très bien coder, elles savent relier un problème à des ressources sur lesquelles elles ont appris et savent bien transposer un problème rencontré dans un autre langage à `Python`.

Mais encore faut-il savoir formuler le problème, savoir juger de la qualité de la réponse du LLM, être capable de remettre en question la proposition de l'assistant pour corriger une erreur ou obtenir une réponse plus satisfaisante. Les LLM sont une recherche *google* très (très) raffinée: si vous n'avez pas les bons mots clés pour faire une recherche Google, votre recherche sera décevante. Il en va de même avec les LLM même si l'aspect conversationnel en langage naturel réduit la barrière à l'entrée.

Le code n'est pas une fin en soi, mais un outil au service d'un raisonnement structuré, visant à résoudre un problème concret.

La confrontation à un jeu de données, à la recherche de réponse non évidentes, va soulever plus ou moins de questions scientifiques et d'ingénierie. Comme un ingénieur qui conçoit un pont, qui va chercher à satisfaire le besoin de traversée avec les connaissances scientifiques et les matériaux disponibles, le *data scientist* part d'un objectif opérationnel (construire un algorithme de sélection, mesurer l'impact d'un lancement produit, prédire une évolution de ventes) pour le formuler de manière exploitable. Cela implique de traduire des concepts scientifiques ou *business* en questions analytiques, puis de décomposer le problème en étapes logiques, chacune traduites en instructions que la machine peut exécuter.

Dans ce cadre, un LLM peut jouer un rôle d'assistant, mais seulement si le problème est bien posé. Si les étapes sont floues ou mal définies, la réponse du modèle sera approximative, voire inutile. Certes la frontière est sans cesse repoussée grâce aux capacités de raisonnement accrues des modèles mais ces outils restent limités. Sur une tâche standard, le résultat pourra sembler correct, mais sur une question plus spécifique, il faudra souvent affiner, reformuler, itérer… et parfois ne jamais obtenir de réponse satisfaisante. Non pas parce que le modèle est mauvais, mais parce que l'ingénierie du problème en amont fait toute la différence[6].

4.2 Car coder, c'est réflechir

De plus, si on ne prend pas le temps, pendant les études, de se confronter à ces problèmes, d'apprendre, quand aura-t-on le temps de le faire ? Bien que nous ayons encore peu de recul sur ce sujet, la recherche en sciences cognitives commence à accumuler des résultats qui montrent un effet

[6] Il n'est pas inutile sur ce sujet de lire le *post* de Thomas Wolf (CSO de `HuggingFace`) « The Einstein AI model ». Même si le *post* évoque principalement les innovations de rupture en mettant quelque peu de côté les innovations marginales, il est intéressant de comprendre que les LLM, en dépit des grandes annonces prophétiques faites par les gourous de la tech, restent des outils qui certes ont de bonnes performances sur des tests standardisés mais restent des assistants, pour l'heure.

négatif de l'utilisation de l'IA sur le cerveau (voir le concept de « dette cognitive » chez N. Kosmyna *et al.* [5]).

Il ne s'agit pas de verser dans la nostalgie de l'époque dorée où on pouvait passer plusieurs heures à se casser la tête sur un problème de code qu'une IA peut maintenant résoudre en quelques minutes voire secondes. J'utilise moi-même quotidiennement des IA génératives pour assister la génération de code ou la création de contenu (par exemple pour la traduction automatique de ce cours en Anglais). Cependant, il ne faut pas oublier que l'objectif est de pouvoir faire plus de *deep work* (et y prendre plaisir!) et non devenir uniquement une machine aliénée, condamnée à cliquer sur le bouton *« Yes »* le plus vite possible lorsque Claude demande une autorisation. C'est difficile puisque les outils disponibles tendent à nous rendre dépendants d'eux. La meilleure manière d'éviter d'être captif de ces outils reste d'acquérir, et de continuer à contruire tout au long de sa vie professionnelle, des compétences générales: c'est l'ambition de ce cours appliqué de *data science*.

4.3 Car coder, c'est actualiser, encore et toujours, ses connaissances

Une dernière raison pour laquelle se contenter d'une IA de code sans recul critique est que ces dernières sont forcément en retard par rapport aux usages puisqu'elles ont été entraînées sur des données passées. L'écosystème `Python` est très dynamique et, même si les IA des principaux fournisseurs de services sont fréquemment réentrainées et peuvent maintenant accéder à internet pour rafraichir leurs connaissances, certaines librairies peuvent rapidement s'imposer dans leur domaine.

Par exemple, l'année dernière, `uv` a connu une adoption rapide, comme `ruff` l'année d'avant. L'existence d'IA génératives ne dispense donc pas, comme avant, d'avoir une veille technique active et d'être vigilant sur l'évolution des pratiques.

5 Objectifs du cours

5.1 Initier à la démarche de la *data science*

Ce cours s'adresse aux praticiens de la *data science*, discipline entendue ici au sens large comme la **combinaison de techniques issues des mathématiques, de la statistique et de l'informatique pour produire de la connaissance utile à partir de données**.

Comme la *data science* n'est pas uniquement une discipline scientique mais vise également à fournir un ensemble d'outils pour répondre à des objectifs opérationnels, l'apprentissage du principal outil nécessaire à l'acquisition de connaissances en *data science*, à savoir le langage `Python`, est également l'occasion d'évoquer la démarche scientifique rigoureuse à adopter face à des données. Ce cours a pour objectif de présenter la démarche face à un jeu de données, les problèmes rencontrés, les solutions pour les surmonter et les implications que ces dernières peuvent avoir. Il ne s'agit donc pas que d'un cours sur un outil technique, désincarné de problématiques scientifiques.

> Faut-il avoir un background en mathématiques pour ce cours ?
>
> Ce cours présuppose qu'on désire faire un usage de `Python` intense en données dans un cadre statistique rigoureux. Il ne revient que de manière secondaire sur les fondements statistiques ou algorithmiques derrière certaines des techniques évoquées, souvent l'objet d'enseignements dédiés, notamment à l'ENSAE.

Ne pas connaître ces notions n'empêche ainsi pas de comprendre le contenu de ce site *web* car les concepts plus avancés sont généralement présentés à part, dans des encadrés dédiés. La facilité d'usage de `Python` évite de devoir programmer soi-même un modèle, ce qui rend possible l'application de modèles dont on n'est pas expert. La connaissance des modèles sera plutôt nécessaire dans l'interprétation des résultats.

Pour autant, même s'il est relativement facile d'utiliser des modèles complexes avec `Python`, il est fort utile d'avoir du recul sur ceux-ci avant de se lancer dans une démarche de modélisation. Il s'agit de l'une des raisons pour lesquelles la modélisation arrive si tardivement dans ce cours: en plus de faire appel à des concepts statistiques avancés, il est nécessaire, pour produire une modélisation pertinente, d'avoir appréhendé les faits stylisés dans nos données. Bien comprendre la structure des données et leur adéquation avec les hypothèses d'un modèle est indispensable pour construire une modélisation de qualité.

5.2 Reproductibilité

Ce cours donne une place centrale à la notion de reproductibilité. Cette exigence se traduit de diverses manières dans cet enseignement, en premier lieu en permettant que tous les exemples et exercices de ce cours soient testés par le biais de *notebooks* `Jupyter`.

> Un *notebook* est un environnement interactif qui permet d'écrire et d'exécuter du code en direct. Il combine, dans un seul document, du texte, du code qui peut être exécuté et dont les sorties s'affichent après calculs. C'est extrêmement pratique pour l'apprentissage du langage `Python`.
>
> Pour plus de détails, consultez la documentation officielle de Jupyter.

L'ensemble du contenu du site *web* est reproductible dans des environnements informatiques divers. Il est bien sûr possible de copier-coller les morceaux de code présents dans ce site, grâce au bouton présent au dessus des exemples de code:

```
x = "Essayez de me copier-coller"
```

Ligne 1
Cliquez sur le bouton pour copier ce contenu et le coller ailleurs.

Néanmoins, comme ce site présente de nombreux exemples, les allers et retours entre un environnement de test de `Python` et celui-ci pourraient être pénibles. Chaque chapitre est donc facilement récupérable sous forme de *notebook* `Jupyter` grâce à des boutons au début de chaque page. Voici, par exemple, ces boutons pour le premier chapitre consacré à `Pandas` :

Les recommandations concernant les environnements à privilégier pour utiliser ces notebooks sont reportées au prochain chapitre.

L'exigence de reproductibilité se manifeste également dans le choix des exemples pris pour ce cours. L'ensemble du contenu de ce site s'appuie sur des données ouvertes, qu'il s'agisse de données françaises (principalement issues de la plateforme centralisatrice `data.gouv` ou du site *web* de l'Insee) ou de données américaines. Les résultats sont donc reproductibles pour quelqu'un disposant d'un environnement identique[7].

> **i** Note

[7] Le fait d'ouvrir les chapitres sous la forme de *notebooks* dans des environnements standardisés, ce qui sera proposé à partir du prochain chapitre, permet d'assurer que vous disposiez d'un environnement contrôlé. Les installations personnelles de `Python` ont toutes les chances d'avoir subies des bidouillages modifiant votre environnement et pouvant provoquer des erreurs inattendues et difficiles à comprendre: ce n'est donc pas un usage recommandé pour ce cours. Comme vous pourrez le découvrir dans le prochain chapitre, les environnements *cloud* offrent un confort en ce qui concerne la standardisation des environnements.

Des chercheurs américains ont pu parler de crise de la reproductibilité dans le domaine du *machine learning* [6]. Les dérives de l'écosystème de la publication scientifique et les enjeux économiques derrière les publications académiques dans le domaine du *machine learning* ont une place privilégiée parmi les facteurs pouvant l'expliquer.

Néanmoins, l'enseignement universitaire porte également une responsabilité dans ce domaine. Les étudiants et chercheurs ne sont pas formés à ces sujets et s'ils n'adoptent pas cette exigence tôt dans leur parcours, ils n'y seront plus forcément incités ultérieurement. Pour cette raison, en plus de former à `Python` et à la *data science*, ce cours introduit à l'usage du logiciel de contrôle de version `Git` dans une partie dédiée.

Tous les projets des élèves doivent être *open source*, ce qui est l'une des meilleures manières, pour un enseignant, de trouver une consigne pour que les élèves produisent un code de qualité.

5.3 Évaluation

Les élèves de l'ENSAE et l'ENSAI valident le cours grâce à un projet approfondi. Les éléments relatifs à l'évaluation du cours, ainsi qu'une liste des projets déjà effectués, sont disponibles dans la section Évaluation.

6 Plan du cours

Ce cours est une introduction aux enjeux de la *data science* à travers l'apprentissage du langage `Python`. Comme le terme *« data science »* l'indique, une partie importante de ce cours est consacrée au travail sur les données: récupération, structuration, exploration, mise en relation.

C'est l'objet de la première partie du cours « Manipuler des données » qui sert de fondement au reste du cours. Malheureusement, de nombreuses formations en *data science*, statistiques appliquées ou sciences économiques et sociales, font l'impasse sur cette part du travail des *data scientists* - qu'on appelle parfois « data wrangling » ou *« feature engineering »* - qui, en plus de représenter une part importante du temps de travail des *data scientists*, est indispensable pour construire un modèle pertinent.

L'objectif de cette partie est d'illustrer les enjeux liés à la récupération de plusieurs types de sources de données et à leur exploitation par le biais de `Python`. Les exemples seront diversifiés, pour illustrer la richesse des données qui peuvent être analysées avec `Python`: statistiques d'émissions communales de CO_2 en France, données de transactions immobilières, diagnostics énergétiques des logements, données de fréquentation des stations vélib…

La deuxième partie est consacrée à la production de visualisations avec `Python`. Après avoir récupéré et nettoyé des données, on désire généralement synthétiser celles-ci par le biais de tableaux, de productions graphiques ou de cartes. Cette partie est une introduction rapide à ce sujet (« Communiquer avec `Python` »). Assez introductive, l'objectif de cette partie est surtout de donner quelques notions qui sont consolidées par la suite.

La troisième partie est consacrée à la modélisation à travers l'exemple fil rouge de la science électorale (« Modéliser avec `Python` »). L'objectif de cette partie est d'illustrer la démarche scientifique du *machine learning*, les choix méthodologiques et techniques afférents et ouvrir vers les enjeux suivants qui seront évoqués dans la suite du cursus universitaire.

La quatrième partie du cours fait un pas de côté pour se consacrer aux enjeux spécifiques liés à l'exploitation des données textuelles. Il s'agit du chapitre d'« Introduction au *Natural Language*

Processing (NLP) » avec `Python` « . Ce champ de recherche étant particulièrement actif, il ne s'agit que d'une introduction au sujet. Pour aller plus loin, se référer à S. J. Russell et P. Norvig [4], chapitre 24.

Ce chapitre propose aussi une partie consacrée au contrôle de version avec le logiciel `Git` (Découvrir `Git`). Pourquoi proposer ce contenu dans le cadre d'un cours de `Python` ? Car apprendre `Git` permettra de produire de meilleurs codes `Python`, de les échanger, voire même de les tester dans des environnements reproductibles ou de mettre à disposition les résultats en ligne (c'est une utilisation plus avancée, objet du cours de mise en production). Ce pas de côté dans l'apprentissage de `Python` est très utile, *a fortiori* dans un monde où `Github` sert de vitrine et où les entreprises et administrations exigent, à juste titre, que leurs *data scientists* sachent faire du `Git`.

De manière plus détaillée, voici le plan du cours:

Introduction

Title	Available in	Learning mode	Resource type
Introduction	France United States	Lesson #1 or self learning	Read from website
Avoir un environnement Python fonctionnel pour la data science	France United States	Lesson #1 or self learning	Read from website and exercise notebooks
Démarche à adopter face à un jeu de données	France United States	Lesson #1 or self learning	Read from website
Quelques exercices de rappels pour se remettre en selle	France United States	Lesson #1 or self learning	Read from website

Manipuler des données

Title	Available in	Learning mode	Resource type
Introduction à la partie 1: manipuler des données	France United States	Lesson #1 or self learning	Read from website
Numpy, la brique de base de la data science	France United States	Lesson #1 or self learning	Read from website and exercise notebooks
Introduction à Pandas	France United States	Lesson #2 or self learning	Read from website and exercise notebooks
Associer des données avec Pandas : les jointures	France United States	Lesson #2 or self learning	Read from website and exercise notebooks

Title	Available in	Learning mode	Resource type
Manipuler des données avec Pandas : statistiques et restructuration	France United States	Lesson #2 or self learning	Read from website and exercise notebooks
Pandas : limites et écosystèmes alternatifs	France United States	Lesson #2 or self learning	Read from website
Introduction aux données spatiales avec Geopandas	France United States	Lesson #4 or self learning	Read from website and exercise notebooks
Web scraping avec Python	France United States	Self learning	Read from website and exercise notebooks
Récupérer des données avec des API depuis Python	France United States	Lesson #4 or self learning	Read from website and exercise notebooks
Maîtriser les expressions régulières	France United States	Self learning	Read from website and exercise notebooks
Les nouveaux modes d'accès aux données : le format Parquet et les données sur le cloud	France United States	Self learning	Read from website

Communiquer

Title	Available in	Learning mode	Resource type
Introduction à la partie 2: communiquer à partir de données	France United States	Self learning	Read from website
Construire des graphiques avec Python	France United States	Self learning	Read from website and exercise notebooks
Introduction à la cartographie avec Python	France United States	Self learning	Read from website and exercise notebooks

Modéliser

Title	Available in	Learning mode	Resource type
Introduction à la partie 3: modéliser	France United States	Lesson #5 or self learning	Read from website
Préparation des données pour construire un modèle	France United States	Lesson #5 or self learning	Read from website and exercise notebooks
Evaluer la qualité d'un modèle	France United States	Lesson #5 or self learning	Read from website and exercise notebooks
Découverte de la classification avec les arbres de décision	France United States	Lesson #5 or self learning	Read from website and exercise notebooks
Introduction à la régression	France United States	Lesson #6 or self learning	Read from website and exercise notebooks
Sélection de variables : une introduction	France United States	Lesson #6 or self learning	Read from website and exercise notebooks
Introduction à l'apprentissage non supervisé avec le clustering	France United States	Lesson #6 or self learning	Read from website and exercise notebooks
Premier pas vers l'industrialisation avec les pipelines scikit	France	Self learning	Read from website and exercise notebooks
Mettre à disposition un modèle par le biais d'une API	France	Self learning	Read from website

Natural Language Processing (NLP)

Title	Available in	Learning mode	Resource type
Introduction à la partie 4 : Natural Language Processing (NLP)	France United States	Self learning	Read from website
Nettoyer et structurer l'information dans les données textuelles	France United States	Lesson #7 or self learning	Read from website and exercise notebooks

Title	Available in	Learning mode	Resource type
L'analyse fréquentiste par l'approche bag-of-words : intérêt et limites	France United States	Lesson #7 or self learning	Read from website and exercise notebooks
Les embeddings, ou comment synthétiser l'information textuelle	France United States	Lesson #7 or self learning	Read from website and exercise notebooks

Git

Title	Available in	Learning mode	Resource type
Git: un outil nécessaire pour les data scientists	France United States	Lesson #3 or self learning	Read from website
Découvrir Git par la pratique: la gymnastique quotidienne	France United States	Lesson #3 or self learning	Read from website
Un cadavre exquis pour découvrir le travail collaboratif `Git`	France United States	Lesson #3 or self learning	Read from website

Références

Informations additionnelles

ℹ Environnement Python

Ce site a été construit automatiquement par le biais d'une action `Github` utilisant le logiciel de publication reproductible `Quarto` (version 1.10.18).

L'environnement utilisé pour obtenir les résultats est reproductible par le biais d'uv. Le fichier `pyproject.toml` utilisé pour construire cet environnement est disponible sur le dépôt `linogaliana/python-datascientist`

```
[project]
name = "python-datascientist"
version = "0.1.0"
description = "Source code for Lino Galiana's Python for data science course"
readme = "README.md"
```

```
requires-python = "⩾3.13,<3.14"
dependencies = [
    "altair⩾6.0.0",
    "cartiflette⩾0.2",
    "contextily==1.6.2",
    "duckdb⩾0.10.1",
    "folium⩾0.19.6",
    "gdal==3.11.4",
    "graphviz==0.20.3",
    "great-tables⩾0.12.0",
    "gt-extras⩾0.0.8",
    "ipykernel⩾6.29.5",
    "jupyter⩾1.1.1",
    "jupyter-cache⩾1.0.0",
    "kaleido⩾0.2.1",
    "langchain-community⩾0.3.27",
    "loguru==0.7.3",
    "markdown⩾3.8",
    "nbclient⩾0.10.0",
    "nbformat⩾5.10.4",
    "nltk⩾3.9.1",
    "openpyxl⩾3.1.5",
    "pandas⩾3.0",
    "pip⩾25.1.1",
    "plotly⩾6.1.2",
    "plotnine⩾0.15",
    "polars⩾1.8.2",
    "py7zr⩾1.1.3",
    "pyarrow⩾17.0.0",
    "pynsee⩾0.1.8",
    "python-dotenv⩾1.0.1",
    "python-frontmatter⩾1.1.0",
    "pywaffle⩾1.1.1",
    "requests⩾2.32.3",
    "scikit-image⩾0.24.0",
    "scikit-learn⩾1.8.0",
    "scipy⩾1.13.0",
    "seaborn⩾0.13.2",
    "selenium<4.39.0",
    "spacy⩾3.8.4",
    "webdriver-manager⩾4.0.2",
    "wordcloud==1.9.3",
]

[tool.uv.sources]
gdal = [
  { index = "gdal-wheels", marker = "sys_platform == 'linux'" },
  { index = "geospatial_wheels", marker = "sys_platform == 'win32'" },
]

[[tool.uv.index]]
name = "geospatial_wheels"
url = "https://nathanjmcdougall.github.io/geospatial-wheels-index/"
explicit = true

[[tool.uv.index]]
name = "gdal-wheels"
```

```
url = "https://gitlab.com/api/v4/projects/61637378/packages/pypi/simple"
explicit = true

[dependency-groups]
dev = [
    "nb-clean>=4.0.1",
]
```

Pour utiliser exactement le même environnement (version de `Python` et *packages*), se reporter à la documentation d'`uv`.

Bibliographie

[1] T. H. Davenport et D. Patil, « Data scientist, the sexiest job of the 21st century », *Harvard business review*, vol. 90, nº 5, p. 70-76, 2012, [En ligne]. Disponible sur: https://hbr.org/2012/10/data-scientist-the-sexiest-job-of-the-21st-century

[2] T. H. Davenport et D. Patil, « Is data scientist still the sexiest job of the 21st century? », *Harvard Business Review*, vol. 90, 2022.

[3] L. Galiana et O. Lefebvre, « Qu'apportent les métiers de la data à la statistique publique ? », *Courrier des Statistiques*, nº 15, p. 95-116, 2026, [En ligne]. Disponible sur: https://insee.hal.science/hal-05684911v1

[4] S. J. Russell et P. Norvig, *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020. [En ligne]. Disponible sur: http://aima.cs.berkeley.edu/

[5] N. Kosmyna *et al.*, « Your brain on ChatGPT: Accumulation of cognitive debt when using an AI assistant for essay writing task », *arXiv preprint arXiv:2506.08872*, vol. 4, p. 206, 2025.

[6] S. Kapoor et A. Narayanan, « Leakage and the Reproducibility Crisis in ML-based Science ». [En ligne]. Disponible sur: https://arxiv.org/abs/2207.07048